

Measures of Query Cost

Core Concept: A query can be executed through multiple evaluation plans. **Query optimization** selects the most efficient one by estimating the **cost** of individual operations and combining them into a total query cost.

1. Dimensions of Query Cost

The cost of executing a query can be evaluated in terms of different resource usages:

- **Disk Access Cost:** Number of blocks read or written from storage.
- **CPU Cost:** Time for processing tuples, evaluating operators, or scanning indices.
- **Communication Cost:** Network overhead in parallel or distributed systems.

Early models emphasized disk I/O cost, as it dominated on magnetic disks. Today, with SSDs and large main memory, CPU costs are also significant.

2. Evolution of Storage and Performance

- Modern databases often reside in **main memory** or **solid-state drives (SSDs)**.
- I/O latency has reduced drastically; **CPU time** has become more relevant.
- Data persistence is still maintained on flash or magnetic storage.

Observation

For in-memory databases, I/O cost no longer dominates. Thus, both CPU and memory performance influence query evaluation cost.

3. PostgreSQL Cost Model (Example)

PostgreSQL estimates query cost by combining CPU and I/O factors:

- i) CPU cost per processed tuple.
- ii) CPU cost per index entry.
- iii) CPU cost per operator or function.

Each cost is multiplied by the respective count (tuples, entries, operations). Default parameters can be modified through PostgreSQL's configuration.

4. Estimating I/O Cost

The two most fundamental parameters are:

t_T = Time to transfer one block, t_S = Average seek or access time.

If an operation transfers b blocks and performs S random I/O accesses:

$$\boxed{\text{Total Cost} = b \times t_T + S \times t_S}$$

Typical (2018) Storage Parameters

- **Magnetic Disk:** $t_S = 4 \text{ ms}$, $t_T = 0.1 \text{ ms}$
- **SATA SSD:** $t_S \approx 90 \text{ } \mu\text{s}$, $t_T \approx 10 \text{ } \mu\text{s}$
- **PCIe 3.0×4 SSD:** $t_S = 20\text{--}60 \text{ } \mu\text{s}$, $t_T \approx 2 \text{ } \mu\text{s}$
- **Main Memory:** $t_S < 100 \text{ ns}$, $t_T < 1 \text{ } \mu\text{s}$

Performance Insights

- SSDs lack physical seeks but exhibit latency for I/O initiation.
- SSDs can achieve $\approx 10,000\text{--}50,000$ random reads per second.
- Main memory access occurs at cache-line level, usually $< 1 \text{ } \mu\text{s}$.

5. Read vs. Write Cost

- Writes are generally slower than reads (due to verification overhead).
- On magnetic disks: write $\approx 2\times$ read cost.
- On PCIe SSDs: write throughput $\approx 50\%$ lower, but masked by interface limits.

6. Buffer and Memory Considerations

Query performance heavily depends on available main memory M .

- If data fits in memory buffers, disk access is minimized.
- PostgreSQL's default *effective cache size* = 4 GB.

PostgreSQL Cache Model:

- Random page read cost = $\frac{1}{10}$ of actual cost (assuming 90% cache hits).
- Internal B⁺-tree nodes are assumed always cached (only leaf I/O counted).

7. Cost Estimation Summary

- **Main Cost Factors:** Disk I/O, CPU, and communication.
- **Cost Metrics:**
 - **Response Time:** Wall-clock time for a query to finish.
 - **Total Resource Consumption:** Total usage of I/O, CPU, and memory.
- **Optimization Focus:** Minimize total resource consumption since response time is difficult to predict.

Simplified Disk Cost Formula

$$\text{Disk Cost} = (\text{Number of Seeks}) \times t_S + (\text{Blocks Read}) \times t_T + (\text{Blocks Written}) \times t_T$$

Assuming equal read/write cost:

$$\boxed{\text{Total Cost} = b \times t_T + S \times t_S}$$

8. Impact of Buffer and Memory

- Some data may already be **buffer-resident**, reducing I/O.
- Memory available to a query depends on concurrent system load.
- Algorithms can trade memory for fewer disk I/Os.
- Optimizers typically assume an available memory size M during estimation.

Estimation Scenarios

- **Worst Case:** Minimal memory and no data cached.
- **Optimistic Case:** Partial or full data already resident in memory.

9. Response Time vs. Resource Use

Response time (wall-clock time) depends on:

1. Current buffer contents when the query starts.
2. Disk distribution and data placement.

Because these are unpredictable during optimization, optimizers minimize **total resource consumption** instead.

Example — Parallel vs. Sequential Access

A plan A may read more data but in parallel across multiple disks, completing faster than plan B , which reads sequentially. However, if many instances of A run simultaneously, disk contention may increase overall response time.

10. Summary

- Query cost reflects combined I/O, CPU, and communication effort.
- Disk parameters (t_S , t_T) remain key for estimation.
- Modern optimizers focus on minimizing total resource usage.
- Memory and caching dramatically affect real execution times.